بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ

In the name of Allah, Most Gracious, Most Merciful

CSE 4107
Structured Programming I

Topic: Expressions

Asaduzzaman Herok
Lecturer | CSE | IUT
asaduzzaman34@iut-dhaka.edu

Expressions

The `expressions`: formulas that show how to compute a value rather than statements

`Follow the white board`

- Simplest expression: Variables (value to be computed), Constants (fixed value)
- Complicated expressions: Apply operators on operands

```
a + (b * c)              //Expression
```

- `a` and `(b*c)` are also expressions
- Building expression: Operators are basic tools
  - Arithmetic operators: +, -, * and /
  - Relational operators: <, <=, ==, >=, >
  - Logical operators: ! (logical negation), && (logical and), || (logical or)

Source: C Programming Modern Approach Book

Arithmetic Operators

The `arithmetic operators`: perform addition, subtraction, multiplication, and division.

Follow the white board

- Unary Operators: requires only one operand.

```
i = +i;   /* + as unary operators */
i = -i;   /* - as unary operators */
```

- Binary Operators: requires two operand.

Unary	***Binary***	
+ unary plus	***Additive***	***Multiplicative***
- unary minus	+ addition	* multiplication
++ increment	- subtraction	/ division
-- decrement		% modulo/reminder

Source: C Programming Modern Approach Book

Arithmetic Operators

Operator Precedence and Associativity:

Follow the white board

```
i + j * k;
/* add i and j then multiply result with k */
/* multiply j and k then add result with i */
```

- Operator Precedence:
  - Highest: `+ -` (unary)
    `* / %`
  - Lowest: `+ -` (binary)

`i + j * k`	is equivalent to	`i + (j * k)`
`-i * -j`	is equivalent to	`(-i) * (-j)`
`+i +j / k`	is equivalent to	`(+i) + (j / k)`

Source: C Programming Modern Approach Book

Arithmetic Operators

Operator Precedence and Associativity:

Follow the white board

- Operator with same precedence:
  - Left associativity: it groups from left to right. (binary arithmetic operators (`*,/,%,+,` and `-`)

`i - j - k` is equivalent to `(i - j) - k`
`i * j / k` is equivalent to `(i * j) / k`

  - Right associativity: it groups from right to left (unary operators)

`- + i` is equivalent to `-(+i)`

Source: C Programming Modern Approach Book

Assignment Operators

Store value in variables:

Follow the white board

- Simple Assignment : `(v=e)` means evaluate expression `e` and copy the value to `v`.

```
i = 5;      /* j is now 5 */
j = i;      /* i is now 5 */
k = 10 * i + j;     /* k is now 55 */
```

- `v` and `e` has different type: `e` is converted to type of `v`
- Side Effects: Most C operators don't modify the operands. Some does.

```
i = j = k = 5;      /* i, j, k is now 5 */
```

Source: C Programming Modern Approach Book

Assignment Operators

Lvalues:

Follow the white board

- Assignment operators requires *lvalues* as it's left operands.
- *Lvalues*: represents an object stored in memory not constant or result of a computation

```
  12 = i;      /* Wrong */
i + j = 0;      /* Wrong */
  -i = j;      /* Wrong */
```

Source: C Programming Modern Approach Book

Assignment Operators

Compound Assignment: shortens statement

Follow the white board

```
i += 2;   /* same as i = i + 2 */;
```

- 9 compound assignment operators in C: including the followings

  `-=` `*=` `/=` `%=`

- Operator precedence problem: `i *= j + k` is not equivalent to `i = i * j + k`

Source: C Programming Modern Approach Book

Increment and Decrement Operators

Follow the white board

```
 i++; ++i;  /* same as i = i + 1 with a difference */;
i--; --i;   /* same as i = i - 1 with a difference */;
```

- Pre-increment: `++i;`
- Post increment: `i++;`
- Pre-decrement: `--i;`
- Post increment: `i--;`

Source: C Programming Modern Approach Book

Expression Evaluation

Once again the Precedence:

Follow the white board

Precedence	*Name*	*Symbol(s)*	*Associativity*
1	increment (postfix) decrement (postfix)	++ --	left
2	increment (prefix) decrement (prefix) unary plus unary minus	++ -- + -	right
3	multiplicative	* / %	left
4	additive	+ -	left
5	assignment	= *= /= %= += -=	right

Source: C Programming Modern Approach Book

Expression Evaluation

Order of Subexpression Evaluation:

Follow the white board

```
a = b += c++ - d + --e / - f;
```

- C doesn't defines the order of subexpression evaluation (Exceptions are there)

```
a = 5;
c = ( b = a + 2) - (a = 1)  /* undefined behavior */
```

Source: C Programming Modern Approach Book

Expression Statements

Any expression with a semicolon at the end.

Follow the white board

```
( a + b ) * ( c - d );
```

- Do nothing Expression

Source: C Programming Modern Approach Book


Q
A


THANK
YOU